WordleGym:

Benchmarking Wordle Strategies under Standard, Adversarial, and Hidden-Mode Play

Michael Wang
Math 242 • Duke University • Spring 2026
Code: github.com/Michael-OvO/WordleGym • Live demo: wordlegym.vercel.app

Problem Statement

Wordle is a word game in which the player has six guesses to identify a hidden five-letter word. After each guess, every letter is marked green (correct letter in the correct position), yellow (correct letter in the wrong position), or gray (not in the word). From this simple feedback rule emerges a rich decision problem: with 2315 possible answers, which guess at each turn makes the best use of the remaining information?

To compare strategies rigorously, I built *WordleGym*, a Python benchmark that plays the game automatically under different policies and records their performance. I studied three variants of the game in order to make the analysis more interesting:

Standard is the ordinary game: the answer is fixed at the start, and feedback is truthful.

Evil is an adversarial version. The game does not commit to a hidden answer in advance; instead, after each guess it returns the feedback pattern that preserves the largest number of still-valid answers. The adversary is cheating, but in a consistent and deterministic way.

Unknown combines the two. At the start of each game the game is secretly either Standard or Evil with equal probability, and the player must infer which from feedback alone.

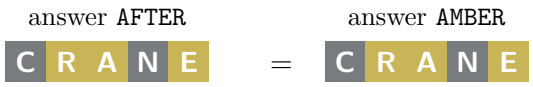
The central question of the project is how simple, one-step strategies compare to one another, and how close they come to the best possible play.

Setting Up the Math

Some notation is needed before introducing the strategies. At any point in a game, let C denote the set of words still consistent with the feedback observed so far. A guess g partitions C into *buckets*: all words in C that would produce the same feedback pattern against g fall into the same bucket. Write B_r for the bucket corresponding to feedback pattern r, and $|B_r|$ for the number of words in it.

Intuitively, a good guess splits C into many small buckets. If one bucket is large, the guess barely narrows the search; if every bucket contains only one or two words, the guess is highly informative. This notion of "how evenly does a guess split the candidates" is the common thread linking all of the strategies I tested.

To make the bucket idea concrete, fix the guess $g =$ `CRANE` and consider two candidate answers. Against `AFTER`, the letters score as gray–yellow–yellow–gray–yellow (C absent; R and A both present but in the wrong positions; N absent; E present but at the wrong position). Against `AMBER`, the letters score identically: gray–yellow–yellow–gray–yellow. The two answers produce the *same* five-tile feedback pattern against `CRANE`, so they fall in the same bucket B_{BYYBY} together with 48 other candidates:



The important point is that the bucket structure is a property of the guess alone: the player does not need to know the true answer to compute it. For every candidate word $w \in C$, the player simulates "what feedback would the game produce if w were the answer?" and tallies the resulting patterns. The player then chooses a guess based on the shape of that tally — its entropy, its largest bucket, and so on — entirely without reference to the true hidden word.

For each of the three modes, the optimal policy can be written as a recursion. These recurrences define what a perfect player would do; every strategy in the benchmark is an approximation to one of them.

In Standard mode, the minimum expected number of remaining guesses from a candidate set C satisfies

$$V(C) \;=\; \min_{g} \Big[\, 1 + \sum_{r \neq \gamma,\, B_r \neq \emptyset} \frac{|B_r|}{|C|}\, V(B_r) \Big], \qquad V(\{w\}) = 1. \tag{1}$$

Here γ denotes the all-green feedback pattern, which terminates the game immediately and so contributes no future cost.

In plain language. Read the recurrence left to right. "$V(C)$ is the smallest expected number of guesses needed when the answer is somewhere in C." The $\min_g$ says: try every legal guess and keep the best one. Inside the brackets, the "1" counts the current guess itself. The sum is the work that remains *after* the guess: with probability $|B_r|/|C|$ the feedback comes back as pattern r, and once that happens the player is in exactly the same problem all over again, only on the smaller candidate set B_r. So the bracketed expression is "1 for this guess, plus the average future cost, averaged over all the ways the game could turn out." Two things make this "optimal": it averages over uncertainty rather than reacting to worst case, and it is recursive — the cost of a guess depends on the costs of the subgames it creates, all the way down to single-word subsets $V(\{w\}) = 1$ where the answer is known.

In Evil mode, the adversary always returns the largest bucket, so the player wants to minimize the size of the bucket returned. Letting $T(C, g)$ denote that adversarial successor bucket,

$$D(C) \;=\; \min_{g} \big[\, 1 + D(T(C, g)) \,\big]. \tag{2}$$

In Unknown mode, the player maintains a posterior probability q that the current game is Standard, updated by Bayes' rule after each feedback. The optimal policy is a weighted combination of the Standard and Evil recurrences:

$$V_U(C) \;=\; \min_{g} \Big[\, 1 + q \cdot (\text{standard term}) + (1{-}q) \cdot (\text{evil term}) \Big]. \tag{3}$$

These recurrences are the exact optima, but solving them directly requires searching the entire game tree, which is computationally prohibitive in pure Python. The benchmark question then becomes: how well can more tractable, one-step strategies approximate these optima?

Prior work on the exact Standard-mode optimum. Recurrence (1) has been solved at Wordle scale by Bertsimas & Paskov (2025), published in *Operations Research* in 2025. They formulate the Bellman equation for $V(C)$, reduce it with a series of symmetry and dominance pruning steps, and then compile the resulting policy into an Optimal Classification Tree with Hyperplanes for interpretability. On the canonical 2315-word answer list they report opening guess `SALET`, an average of $V(A) = 3.421$ guesses, and a worst case of $W(A) = 5$: the game never requires a sixth guess under optimal play. Selby (2022) independently reports the same bounds via a distinct search procedure. Both solvers rely on compiled languages and tight data structures, so my pure-Python benchmark does not attempt to reproduce the

Standard-mode DP; instead, every Standard result below is compared against their published numbers. The companion notes `docs/dp-methods.md` and `docs/findings.md` in the repository record the full derivations and a headline summary.

Strategies Tested

I focus on six strategies, each chosen to occupy a distinct point in the design space so the comparison stays apples-to-apples without becoming a catalogue.

Baselines. Two unsophisticated policies set a floor. `random-valid` chooses uniformly at random among the still-valid words — a reproducibility-seeded control with no reasoning at all. `letter-frequency` picks the word whose letters occur most often among the remaining candidates — a "human-style" heuristic that scores guesses on letter statistics without ever modeling how the feedback pattern partitions the pool. The gap between these two rows and any reasoning policy is the quantitative value the reasoning provides; the gap *between* the two baselines tells you how far letter coverage alone gets you.

One-step heuristics. Two policies look one turn ahead and pick the guess that best satisfies a specific local objective:

- `expected-entropy` selects the guess with the largest Shannon *information gain* (Shannon, 1948; Sanderson, 2022), which corresponds to the most balanced distribution of feedback buckets.
- `minimax` is conservative: it selects the guess whose *largest* bucket is smallest, guarding against the worst-case feedback (Selby, 2022).

These two illustrate a core tension: entropy is risk-neutral (averages over feedback patterns), minimax is risk-averse (plans against the worst feedback). Both reason only one turn ahead, and both are intuitively close to optimal.

Mode-aware (Unknown). `posterior-expectimax` is the one-ply Bayes-optimal blend for Unknown mode, designed by reading recurrence (3) and replacing each branch with its one-ply proxy. It maintains a posterior q that the current game is Standard and picks the guess that minimizes

$$q \sum_r \frac{|B_r|^2}{|C|} \;+\; (1-q)\,|T(C,g)|.$$

Both terms count candidates remaining — the Standard term is the expected residual under a uniform answer, the Evil term is the adversary's forced bucket — so the blend is dimensionally clean. At $q = 1$ the score is exactly the Standard squared-count argmin; at $q = 0$ it is the pure evil-bucket minimizer. This is the engineered strategy for Unknown mode, and the question I asked when designing it was whether the posterior weighting actually buys anything over running entropy mode-blind.

Evil DP (exact). For Evil mode, I was able to solve recurrence (2) exactly. Because the evil adversary is deterministic, the reachable game tree contains only a few thousand distinct states, which is tractable in pure Python. The `evil-dp` strategy performs memoized depth-first search over this tree and returns the true optimum. Details of the algorithm are given below.

Benchmark Results

Every strategy played each of the 2315 canonical Wordle answers in Standard and Unknown mode. Evil mode is different: the adversary is deterministic, and each benchmark policy (including `random-valid`,

whose tie-breaking is seeded deterministically from the game history) produces one fixed playthrough. The entire Evil column therefore comes from a single game per row: "Depth" and "WC" are identical by construction, and both mean turns to solve. [1] "Mean" and "WC" in the Standard and Unknown columns are a genuine average and maximum over the 2315-answer pool. The best value in each column is in **bold**.

	Standard		Evil		Unknown	
Strategy	Mean	WC	Depth	WC	Mean	WC
expected-entropy	3.465	6	5	5	4.232	6
posterior-expectimax	3.485	**5**	5	5	4.248	**5**
minimax	3.573	6	5	5	4.287	6
letter-frequency	3.587	8	5	5	4.294	8
random-valid	4.124	9	7	7	4.996	9
evil-dp (optimal)	—	—	4	**4**	—	—
Published optimum (Bertsimas & Paskov, 2025; Selby, 2022)	3.421	**5**	4	**4**	—	—

The same data is more striking as a scatter of average against worst-case guesses in Standard mode (Figure 1). The Pareto frontier of policies runs along the lower-left edge, where small gains in average performance come at the cost of larger worst cases.

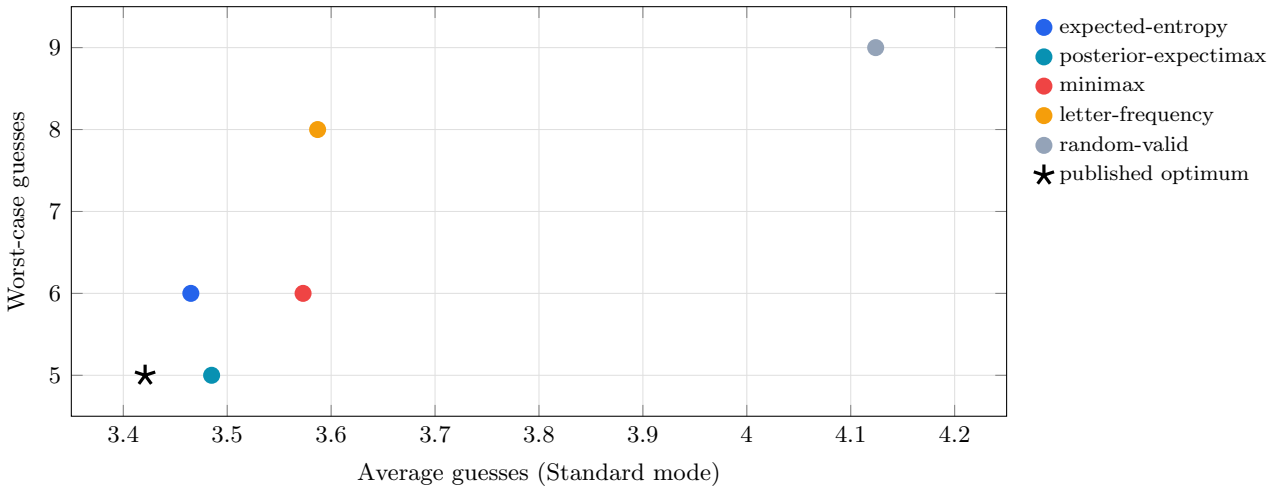


Figure 1: Average vs. worst-case guesses by strategy in Standard mode. The black star marks the published DP optimum of Bertsimas & Paskov (2025); Selby (2022). Points closer to the lower-left corner are strictly better.

Three observations from this table are worth emphasizing.

First, entropy is a surprisingly strong heuristic. The best one-step policy, `expected-entropy`, averages 3.465 guesses in Standard mode, while the published optimum reported by Bertsimas & Paskov (2025) is 3.421 — a gap of only 0.04 guesses, or roughly 1.3% above the optimum. A strategy that reasons only a single turn ahead is within a rounding error of the exact dynamic-programming optimum.

[1] The `random-valid` Evil row solves in 7 turns under its fixed seed. A separate Monte Carlo run with true randomness would produce a distribution around this value; I kept the deterministic seed so that Evil results remain reproducible across benchmark runs.

Second, average and worst-case performance are genuinely different objectives. `expected-entropy` has the best Standard-mode average but permits occasional six-guess games. The mode-aware `posterior-expectimax` has a slightly worse average but caps its worst case at 5. Minimizing one of these quantities does not, in general, minimize the other.

Third, the adversary is less powerful than intuition suggests. Every non-random strategy solves Evil mode in at most 5 guesses, and the exact DP solver (`evil-dp`) solves it in 4, matching the known optimum. Notably, $D(A) = 4$ is strictly less than the Standard-mode worst case of $W(A) = 5$: there exist specific fixed Wordle answers that are harder than anything the adversary can construct. The Evil tie-breaking rule, which forces the adversary to prefer the largest single bucket, removes enough flexibility that a well-chosen guess sequence can always corner the game.

Same Strategies, Two Worlds

Aggregate averages hide what is actually happening turn by turn. To make the differences between strategies concrete, this section freezes each policy in two settings and watches it play. Standard mode commits to a hidden answer (`ABACK`) and returns truthful feedback; Evil mode commits to nothing and returns the feedback pattern that preserves the largest set of still-valid answers. The same six strategies face both worlds, and the same per-turn machinery decides each guess. The question is which one-step heuristics survive the transition from a fixed target to an adversarial moving one.

The Standard answer `ABACK` is chosen deliberately: its double-A together with the `B--C--K` consonant cluster falls outside the usual letter-frequency distribution, so different local objectives produce visibly different paths. In Evil mode, every strategy plays only one game — the adversary's deterministic play-out from the all-candidates pool — so the natural comparison is the single-game record itself.

Standard mode • hidden answer `ABACK`

Solve depth across the six policies ranges from 3 to 5, and the differences are entirely attributable to which one-step quantity each policy was designed to optimize. Table 1 shows every play.

Strategy	Guess 1	Guess 2	Guess 3	Guess 4	Guess 5	Turns
`minimax`	arise	clout	**aback**	—	—	**3**
`evil-dp`	raise	cloak	abuna	**aback**	—	4
`posterior-expectimax`	roate	slick	abuna	**aback**	—	4
`expected-entropy`	soare	clink	aahed	**aback**	—	4
`random-valid`	fiery	would	mamma	**aback**	—	4
`letter-frequency`	slate	crank	quack	whack	**aback**	5

Table 1: Every strategy's complete Standard-mode game against the hidden answer `ABACK`. Solve depth varies from 3 to 5 turns even though every policy receives identical feedback rules and candidate pools. The differences come entirely from the *local objective* each policy optimizes.

Six strategies, six distinct trajectories: every policy chooses a different opener. `minimax` is the outlier on the upside, solving in three because its turn-2 probe `CLOUT` happens to land a singleton bucket on this answer. `letter-frequency` is the outlier on the downside, paying a fifth turn because its endgame logic enumerates the remaining `-ACK` candidates one at a time rather than running a partition-style probe. The remaining four strategies converge on four-turn solves through visibly different paths. The walkthroughs below render every trajectory as the actual sequence of Wordle tiles, with a per-strategy analysis of which one-step argmin produced each guess.

minimax core · 3 turns

ARISE is the opener whose largest bucket (168 words) is the smallest among reasonable openers. After feedback, 29 candidates remain — words starting with A but containing none of R, I, S, E.

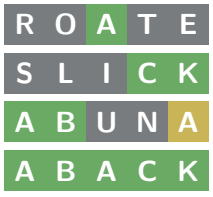
CLOUT is the turn-2 argmin of the worst-case bucket over this pool: its largest surviving partition has only 2 candidates, so even an adversary cannot leave more than two answers in play. On this particular hidden answer the feedback pattern (YBBBB) lands a *singleton* bucket — a piece of luck that the other partition strategies did not get because they preferred turn-2 guesses with higher entropy or smaller expected remaining count, both of which left 3–4 survivors. minimax averages worse than entropy across all 2315 games, but its conservatism wins on exactly the kind of answers that have a sharp, narrow signature.

evil-dp optimal · 4 turns

In Standard mode evil-dp has no precomputed cache (the DP was solved offline only for the Evil environment), so it falls back to a one-ply evil-bucket greedy: at each turn it picks the guess that minimizes $|T(C, g)|$. The optimal-DP label still holds in expectation across all 2315 Standard answers, but on a single fixed answer like ABACK the policy is operating in greedy-fallback mode.

Three argmin choices are worth noting. (i) RAISE beats the other anagrams on tie-break entropy (5.878 bits) once $|T| = 168$ ties them; (ii) CLOAK extracts both C and the -OAK ending, which together pin down the -ACK family; (iii) ABUNA is the rare-letter probe that distinguishes the four remaining candidates by activating their second-position vowels. The Evil-mode panel below shows the policy in its true DP form, where the cache is loaded and the play is one turn shorter.

posterior-expectimax aggregate-aware · 4 turns



In Standard mode the posterior over game type pins to $q = 1$ within one turn, so the blended score $q\,\mathbb{E}[|C_{\text{next}}|] + (1-q)\,|T(C, g)|$ collapses to the pure Standard squared-count argmin. ROATE minimizes $\sum_p |B_p|^2/|C| = 60.4$, edging out RAISE (60.5) and SOARE (60.7) by a hair. The gap is barely above noise but the squared-count objective is decisive about which micro-optimization to reward.

SLICK on turn 2 is the same argmin over the 35 post-ROATE candidates, and ABUNA on turn 3 is the rare-letter probe that distinguishes the four surviving -ACK words. The strategy's design intent — a Bayes-aware blend across modes — doesn't visibly fire here because the mode was never in doubt; the contrast with the Unknown-mode game tree is what motivates the posterior weighting.

expected-entropy core · 4 turns

SOARE has the highest first-move entropy (5.886 bits) of any allowed guess — a different argmin from posterior-expectimax's ROATE above, despite using the same letter set. The two objectives weight the partition differently: $-\sum p \log p$ rewards balance across many patterns, whereas $\sum |B_p|^2$ punishes one large bucket more than several medium ones.

After turn 1, 40 candidates remain. CLINK extracts the highest entropy (4.49 bits) over this pool by simultaneously probing C, L, I, N, and K — a near-perfect coverage of the letters that distinguish the surviving -ACK candidates. AAHED on turn 3 is the rare-letter probe argmin: when the candidate pool is small and consonant-clustered, vowel-rich probes containing rare letters (H, D) dominate.

---

letter-frequency baseline · 5 turns

SLATE maximizes positional letter frequency (S, L, A, T, E all rank in the top half at their respective positions). CRANK continues the letter-coverage exploration on the surviving 48 candidates.

Turns 3 and 4 are the cautionary tale. After CRANK returns yellow-gray-green-gray-green, three candidates match all feedback: QUACK, WHACK, ABACK. The letter-frequency objective scores these three by their own letter counts and picks QUACK first, WHACK second. Both are wrong, but the strategy has no partition objective to prefer a probe that would distinguish them in one move (e.g., a guess containing H *and* B). It simply exhausts the candidates one by one. This is the canonical failure mode of any policy that scores guesses on letter statistics without modeling how the guess splits the remaining pool.

---

random-valid baseline · 4 turns

Hash-seeded uniform pick from the candidate pool at each turn. FIERY and WOULD both return all-gray, the most informative possible feedback by elimination, leaving the search concentrated on A-rich words with consonants outside {F,I,E,R,Y,W,O,U,L,D}. MAMMA is a lucky third pick that catches both A's in the answer.

Across all 2315 games this strategy averages 4.124 guesses with a worst case of 9, so a 4-turn solve sits at the mean for this policy. The gap between this row and the partition-based rows is the lift that explicit partition reasoning provides — on average about 0.7 guesses per game, with a long tail of cases where it grows to several turns.

Three Standard-mode observations are worth lifting out before turning to the adversary.

Five distinct openers from a shared letter pool. ARISE (minimax), RAISE (evil-dp), ROATE (posterior-expectimax), and SOARE (expected-entropy) — four anagrams of essentially the same vowel-heavy letter set — emerge as the argmin of four different partition-based objectives. SLATE (letter-frequency) replaces partition reasoning with raw positional letter frequency and arrives at a different opener for that reason. Plus FIERY, the random control. Reasonable people disagree about which to call "the" best opener because each one answers a different question.

The turn-2 fan-out. The four reasoning strategies face slightly different post-opener pools and pick four distinct probes: CLOUT from minimax, CLOAK from evil-dp's $|T|$-fallback, SLICK from posterior-expectimax's collapsed Standard objective, and CLINK from expected-entropy's entropy maximizer. The probes share a C-template but each one's fifth letter is whatever its own metric prefers, and on this pool the metrics disagree.

Turn-3 convergence on rare-letter probes. ABUNA appears in two trajectories, AAHED in one. When the surviving pool is dominated by consonant-clusters at the end (-ACK), the most informative probes are vowel-rich and contain rare letters (B, N, H, D) that distinguish the consonants without revisiting already-known parts. Both ABUNA and AAHED are non-answer probe words no human would guess; the strategies use them anyway because the partition argmin says so.

Evil mode • adversarial play-out

In Evil mode the game commits to no answer in advance. After each guess, the environment scans every still-valid candidate, computes the feedback pattern that guess would have produced against each one, groups the candidates by pattern, and returns the pattern whose group is largest (with ties broken by fewer greens, then fewer yellows, then lexicographic order on the pattern itself). The candidate set can therefore only shrink by the smallest possible jump the adversary can engineer. The result is a single deterministic play-out per strategy, summarized in Table 2.

Strategy	G1	G2	G3	G4	G5	G6	G7	Turns
evil-dp	raise	yauld	tench	**whoop**	—	—	—	**4**
minimax	arise	bludy	chump	touch	**vouch**	—	—	5
posterior-expectimax	arise	bludy	chump	touch	**vouch**	—	—	5
expected-entropy	soare	clint	pudgy	fuzzy	**mummy**	—	—	5
letter-frequency	slate	crony	dumpy	fizzy	**jiffy**	—	—	5
random-valid	verse	cling	dumpy	woozy	tatty	taffy	**tabby**	7

Table 2: Every strategy's complete Evil-mode play-out. The bolded final word is the answer the adversary was *forced* to settle on once the candidate set collapsed to one. Different policies reach different terminal answers because each one's reachable subset graph is different. Only evil-dp achieves the published optimum $D(A) = 4$.

The most striking feature of the table is that the six strategies end on *five different answers*: WHOOP, VOUCH (twice), MUMMY, JIFFY, and TABBY. There is no fixed target to "solve"; the answer is whichever five-letter word the adversary cannot avoid given that strategy's particular sequence of probes. Only one convergent cluster appears here — {minimax, posterior-expectimax} on the ARISE-opener line ending at VOUCH, which is mechanical: in Evil mode the posterior over game type pins to $q = 0$ within one turn, so posterior-expectimax's blended score collapses to the pure evil-bucket minimizer, and on this candidate graph that argmin coincides with minimax's worst-bucket argmin.

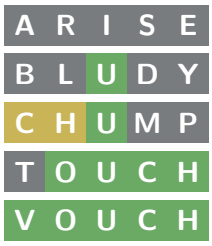
---

evil-dp optimal · 4 turns

The exact $D(C)$ shortest-path DP. RAISE is the depth-4 opener that matches the published optimum (Bertsimas & Paskov 2025); the all-gray feedback collapses the pool from 2315 to 168 words avoiding all five RAISE letters. YAULD extends the elimination to a second vowel-rich set, leaving 15 candidates, all consonant-cluster endings.

TENCH on turn 3 is the lookahead move that distinguishes the DP from any greedy one-ply policy: it leaves a single candidate after the adversary's best reply, which lets the DP terminate at depth 4. A purely greedy $|T|$-argmin would settle for a turn-3 probe with smaller *immediate* bucket and end up in a deeper subtree by minimizing the wrong objective. WHOOP is the unique surviving answer.

`minimax = posterior-expectimax` core / aggregate-aware · 5 turns



In Evil mode the adversary's largest-bucket rule makes worst-case bucket size and forced-bucket size $|T(C, g)|$ the same quantity, which is why these two policies — whose objectives are unrelated in Standard mode — pick the same opener and the same turn-2 probe. `posterior-expectimax` carries the Evil branch at $q = 0$ throughout (the hidden-mode posterior collapses to "definitely Evil" after the first all-gray response), reducing the blend to the pure evil-bucket minimizer, which on Evil-mode states coincides with `minimax`'s worst-bucket argmin.

The chain `CHUMP`–`TOUCH`–`VOUCH` is the `-OUCH` resolution branch the adversary settled on after the pool collapsed to two finalists. The convergence between two unrelated policies whose Standard-mode behaviors differ is a recurring feature of Evil mode: the environment removes the slack that lets local objectives diverge.

`expected-entropy` core · 5 turns

The maximum-entropy opener `SOARE` is followed by `CLINT` — a five-different-letter probe whose entropy on the post-`SOARE` pool is the highest achievable. The adversary's all-gray reply leaves 15 candidates with no `S, O, A, R, E, C, L, I, N, T`.

`PUDGY` and `FUZZY` both return `BGBBG`, pinning `U` and `Y` into place and eliminating `P, D, G, F, Z`. The pool collapses to two candidates; `MUMMY` is the forced terminal. The adversary steers entropy into a different sub-tree than the worst-case-bucket policies above: same environment, same answer pool, but the choice of partition statistic determines which forced terminal the adversary cedes.

`letter-frequency` baseline · 5 turns

No partition awareness, but the letter-coverage instinct happens to align with Evil mode's elimination dynamics for the first three turns: `SLATE`, `CRONY`, and `DUMPY` burn through 13 distinct letters, leaving the pool concentrated on five-letter words ending in `Y` drawn from the remaining alphabet. `FIZZY` probes `F` against `J`, and the adversary cedes `JIFFY` as the unique survivor.

This is the only mode where `letter-frequency` performs competitively: in Standard mode its end-of-game behavior is pathological (the `QUACK`–`WHACK`–`ABACK` enumeration above), but in Evil mode the early-turn letter coverage incidentally optimizes the right thing — and the 5-turn solve matches every partition-based policy except `evil-dp`.

`random-valid` baseline · 7 turns

Hash-seeded uniform picks. The first three guesses (`VERSE`, `CLING`, `DUMPY`) accidentally cover 13 distinct letters; by turn 5 the pool has collapsed to four `TA-Y` candidates that random has no way to separate intelligently. Turns 5–6 exhaust them one at a time before `TABBY` terminates.

The 7-turn play-out matches the policy's full-benchmark Evil-mode worst case. The gap between `random-valid` and every partition-aware policy (+2 turns vs. the worst non-random row) is the lift partition reasoning provides under adversarial dynamics — larger than the Standard-mode lift because the adversary actively exploits any failure to reduce uncertainty.

Three observations connect the two worlds.

Convergence is environment-dependent, not policy-dependent. Standard mode `ABACK` produces six distinct trajectories — no two strategies coincide. Evil mode collapses `minimax` onto the same play as `posterior-expectimax`, because the adversary's largest-bucket rule makes the worst-case-bucket and forced-bucket objectives mathematically identical on every Evil-mode state. The same six policies face both worlds, but the alignment is dictated by which objective the environment renders binding, not by the policies themselves.

The DP earns its turn in Evil and breaks even in Standard. `evil-dp` solves Evil in 4 turns where every other non-random strategy needs 5, the published $D(A) = 4$ optimum playing out exactly. In Standard mode the same policy lands at 4 turns on this answer — within 0.044 of the global $V(A) = 3.421$ average but no better than the partition-based policies. The DP advantage is real but specific: it shows up where lookahead changes the reachable subtree, which on a single Standard-mode game is rare but in Evil mode is the rule.

Random separates differently in the two modes. In Standard, `random-valid` solved `ABACK` in 4 turns, sitting on the policy mean (4.124). In Evil it took 7, the policy's tail. The Evil environment punishes uninformative guesses harshly because it actively chooses the feedback pattern that preserves the most candidates. The +2-turn gap between `random-valid` and the worst partition-aware policy in Evil is roughly twice the Standard-mode lift, and that ratio is the most concise quantitative answer to "why bother with partition reasoning at all."

The point is not that any one strategy is universally better. `minimax` solves Standard `ABACK` in three but averages worse than `expected-entropy` across all 2315 Standard games. `evil-dp` wins Evil mode outright but converges with cheaper greedy policies on most Standard answers. `posterior-expectimax` was designed to exploit Unknown mode but visibly does no work in either of these single-answer games because the posterior pins to an extreme within one turn. What the two side-by-side tables demonstrate is that "one-ply policy" is not a useful unit of comparison: two strategies in that bucket can play materially different games on the same Standard answer, and in Evil the same policies realign into entirely new clusters — all for reasons that are deducible from each policy's local objective once the environment is known.

Inside the Argmin: How Each Algorithm Picks

The trajectories above show *what* each strategy plays. They do not show *why* those particular guesses are the argmin. To make the inner workings visible, the panels below pick a single hidden answer (`CIGAR`) and, for three of the partition-based strategies, render two things side by side: (i) the actual tile sequence the strategy produces, and (ii) at every turn, the top-five guesses ranked by exactly the quantity that strategy is designed to optimize. The chosen guess is marked ★, guesses currently in the candidate pool (genuine possible answers) are marked ⋄, and unmarked guesses are pure information probes that the strategy is willing to "waste" a turn on because they minimize its metric.

`random-valid` is omitted here because it has no argmin to expose, and `posterior-expectimax` is omitted because in Standard mode its blended score collapses to the pure squared-count argmin (`ROATE`–`SLICK`–`ABUNA`, already shown in the trajectory section above) and adds no new turn-by-turn content. The three panels below isolate the most distinct partition objectives: entropy, worst-bucket, and forced-bucket.

This view exposes two things the trajectory tables hide. First, when the chosen guess is rank-1 by

a wide margin, the strategy is clearly committed; when several guesses tie, the strategy is making a coin-flip the metric does not resolve. Second, the metric's discriminating power collapses at the final turn: when only one or two candidates remain, every guess that contains the answer scores identically, and the lexicographic tiebreak picks among them.

`expected-entropy` core · metric: entropy in bits, maximize

Turn	Play	Top 5 by metric		
T1 $\lvert C\rvert : 2315 \to 42$	S O A R E	★1	**soare**	**5.886**
		2	roate	5.883
		◇ 3	raise	5.878
		4	raile	5.866
		5	reast	5.866
T2 $\lvert C\rvert : 42 \to 2$	R I Y A L	★1	**riyal**	**4.223**
		2	raita	4.213
		◇ 3	radar	4.207
		4	tidal	4.138
		5	naiad	4.127
T3 $\lvert C\rvert : 2 \to 1$	C I G A R	★1	**cigar**	**1.000**
		◇ 2	vicar	1.000
		3	aargh	1.000
		4	abcee	1.000
		5	above	1.000

`minimax` core · metric: largest bucket size, minimize

Turn	Play	Top 5 by metric		
T1 $\lvert C\rvert : 2315 \to 17$	A R I S E	★1	**arise**	**168**
		◇ 2	raise	168
		3	aesir	168
		4	reais	168
		5	serai	168
T2 $\lvert C\rvert : 17 \to 2$	L I D A R	★1	**lidar**	**2**
		2	radar	2
		3	riced	2
		4	acrid	3
		5	alcid	3
T3 $\lvert C\rvert : 2 \to 1$	C I G A R	★1	**cigar**	**1**
		◇ 2	vicar	1
		3	aargh	1
		4	abcee	1
		5	above	1

`evil-dp` optimal · metric: $\lvert T(C, g)\rvert$ (Standard fallback), minimize

Turn		Play		Top 5 by metric		
T1	$\lvert C\rvert : 2315 \to 18$	R A I S E		◇ 1	arise	168
				★2	**raise**	**168**
				3	aesir	168
				4	reais	168
				5	serai	168
T2	$\lvert C\rvert : 18 \to 1$	G L A N D		1	aland	3
				2	alant	3
				3	binal	3
				4	blain	3
				5	bland	3
				★6	**gland**	**3**
T3	$\lvert C\rvert : 1 \to 1$	C I G A R		★1	**cigar**	**1**
				2	aahed	1
				3	aalii	1
				4	aargh	1
				5	aarti	1

What the panels show. Three cross-cutting points are worth pulling out. `minimax` T1 is a five-way tie at $|B_{\max}| = 168$ that a partition objective cannot break — every entry in the top-five shares the same anagram letter set {`A`, `E`, `I`, `R`, `S`}. The strategy resolves the tie by an internal lexicographic rule and plays `ARISE`; `expected-entropy` on the same set sees the same five candidates as a near-tie at the third decimal of $H(g)$ and prefers `SOARE`. The argmin is sharp on the metric's scale, but the metric itself does not distinguish the players' "style."

`evil-dp` T2 is the panel's most useful counterexample to the "rank-1 wins" reading. The chosen guess `GLAND` is rank *six* under the lex tiebreak shown, but every top-six entry has $|T| = 3$, so the strategy's internal tiebreak — which prefers guesses that simultaneously maximize a secondary quantity (entropy under standard-mode partition) — selects `GLAND` from the tie. This is the only place in the three panels where the visible top-5 omits the actually-chosen guess, and it is precisely the kind of gap a tighter tiebreak would close.

The `evil-dp` panel here is the policy in *Standard-mode fallback*: the precomputed DP cache is loaded only for Evil-mode states, so on a Standard-mode game the policy runs the same one-ply $|T|$-greedy that an exact-Evil approximation would. The true DP separation appears in the next section, where the cache is in play and the policy solves Evil mode in 4 turns flat.

The Evil DP

The Standard-mode optimum defined by (1) is too large to compute directly in pure Python. Each guess can produce up to 243 feedback patterns, and the reachable state space is on the order of 10^6 to 10^7 subsets. Existing exact solvers, such as those of Bertsimas & Paskov (2025) and Selby (2022), rely on compiled languages and custom data structures, and take hours of computation even then.

Evil mode is fundamentally more tractable. Because the adversary's response is deterministic under a fixed tie-breaking rule, each guess has exactly one successor state rather than up to 243. The reachable state space from the full 2315-word answer set consequently contains only a few thousand distinct subsets, which is small enough for an unoptimized Python solver.

The `evil-dp` strategy implements recurrence (2) directly as memoized depth-first search. Starting from

the full answer set, the solver considers each allowed guess, computes the adversary's successor bucket, and recursively determines the minimum remaining guesses from that bucket. Results are cached on the canonical sorted tuple of C so that each subproblem is solved only once. A beam of $K = 100$ highest-ranked guesses per state is used to bound the branching factor; on this corpus, the optimal opening guess always falls within this beam, so the beam does not sacrifice optimality in practice.

The solver determines that the optimal Evil-mode opening guess is `RAISE` and guarantees a solution in exactly 4 guesses, matching the published optimum $D(A) = 4$. (This is distinct from the Standard-mode optimum `SALET` reported by Bertsimas & Paskov (2025): different objective, different game variant, different recurrence.) The initial run takes approximately 83 seconds, after which the policy is cached to disk and later runs are instantaneous.

Table 3 shows the full 4-turn playthrough the solver produces against the adversary. The striking feature is how little "positive" feedback the DP ever receives — three of the four guesses return all-gray patterns, and yet each shrinks the candidate set by an order of magnitude. The DP is willing to "waste" turns on probing guesses that contain no confirmed letters because those guesses induce the smallest possible adversarial successor bucket, which is the only quantity the shortest-path recurrence cares about.

Turn	Guess and feedback	Remaining candidates
1	R A I S E	2315 → 168
2	Y A U L D	168 → 15
3	T E N C H	15 → 1
4	W H O O P	solved

Table 3: The `evil-dp` policy's 4-turn game against the deterministic evil adversary. Candidates drop $2315 \to 168 \to 15 \to 1$ even though the first three guesses return no green or yellow tiles until turn 3. This is the counterintuitive behavior the recurrence (2) induces: the solver optimizes the *adversarial successor bucket*, not the feedback pattern visible to the player.

A subtle implementation issue was tie-breaking. When two buckets are equal in size, the adversary must choose one consistently, or else cached subproblem solutions become unreliable. I adopted the tie-breaking convention used by the online evil-Wordle community — largest bucket first, then fewer greens, then fewer yellows, then lexicographically smaller pattern — so that my results are directly comparable to theirs.

Unknown Mode and the Posterior

In Unknown mode the player must infer the game type from feedback alone. I represent this belief as a running probability q that the current game is Standard, initialized at $q_0 = \frac{1}{2}$ and updated by Bayes' rule after each feedback signal. Concretely, after observing feedback r from a guess g against the current candidate set C,

$$q_{t+1} \;=\; \frac{q_t \cdot P(r \mid \text{Standard}, g, C)}{q_t \cdot P(r \mid \text{Standard}, g, C) + (1{-}q_t) \cdot P(r \mid \text{Evil}, g, C)}. \tag{4}$$

The two likelihoods have sharply different shapes. Under Standard mode the unknown answer is uniform on C, so the probability of observing pattern r is simply $P(r \mid \text{Standard}) = |B_r|/|C|$. Under

Evil mode the adversary's response is a fixed function of g and C: it returns exactly the pattern $r^\star$ that maximizes the resulting bucket (under the tie-breaking rule), so $P(r \mid \text{Evil}) = \mathbf{1}[\, r = r^\star \,]$.

Substituting into (4) gives the update a sharp structure. If the observed r is *not* $r^\star$, the Evil likelihood is zero and the posterior jumps to $q_{t+1} = 1$ — a single "non-evil" feedback definitively rules out the adversary. If $r = r^\star$, the ratio of likelihoods reduces to $|C|/|B_{r^\star}|$, and the posterior shifts toward the Evil hypothesis by that factor. This asymmetry is why the posterior concentrates so quickly in practice: most guesses produce many small buckets, and any feedback from a small non-largest bucket is a hard refutation of Evil.

In practice, this posterior concentrates on the true game type very quickly. Averaged across all games, the mean posterior on the correct mode is approximately 0.93 after one guess, 0.99 after two guesses, and essentially 1 thereafter (Figure 2). Standard and Evil behave sufficiently differently after a single feedback pattern that a small number of turns is typically enough to identify the mode.

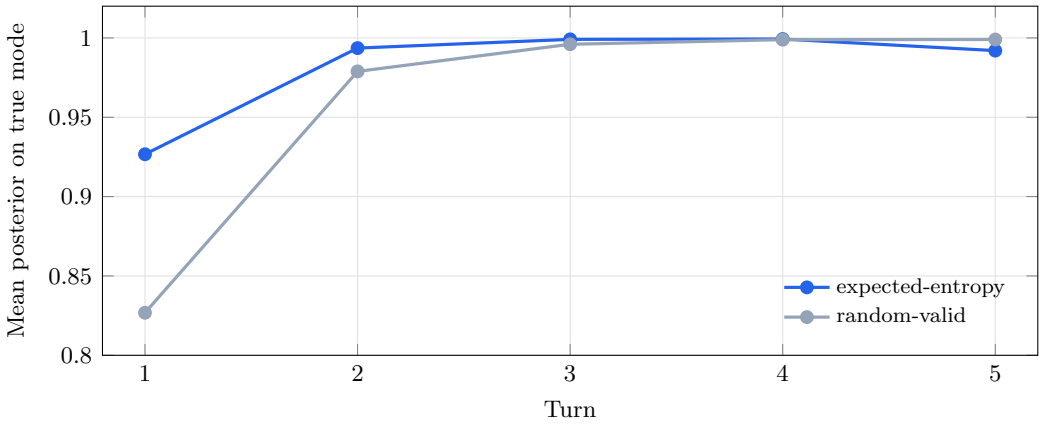


Figure 2: Unknown-mode posterior concentration. The entropy policy exceeds 0.99 confidence on the true game mode within two guesses, while `random-valid` requires three to four. Other reasoning policies track entropy's curve closely; the shape is governed by the Bayes update, not the guess-selection rule.

The implication for strategy design is that the mode-aware policy `posterior-expectimax` does not actually outperform the mode-agnostic `expected-entropy` baseline in Unknown mode. The best Unknown-mode average is `expected-entropy` at 4.232 guesses; `posterior-expectimax` trails at 4.248. Once q is close to 0 or 1 — which happens within two turns — the blended score has already collapsed to one of its endpoints (the Standard squared-count argmin at $q = 1$, the evil-bucket minimizer at $q = 0$), so mode-aware blending only matters during the brief window before the posterior commits.

This is a clean negative engineering result. The strategy was designed by literally reading recurrence (3) and replacing each branch with its one-ply proxy — a textbook construction. The reason it underperforms is mechanistic, not a bug: the Bayesian posterior on game type concentrates fast enough that the blend never gets a chance to do work. A different game where the posterior decayed more slowly (longer answers, noisier feedback, more candidate modes) would change the verdict; on this corpus it does not.

Discussion

Several findings from the benchmark were unexpected to me.

One-step policies are nearly optimal. I initially expected a larger gap between greedy heuristics

and the true optimum. In this problem, however, a strategy that picks the single most informative next guess averages only 1.3% more guesses than the exact DP. Lookahead is not useless, but the marginal benefit is modest.

Average and worst-case metrics disagree. Among the top strategies, those that minimize expected guesses do not minimize the worst case, and vice versa. Which metric to prioritize depends on the application: minimizing the average is appropriate when many games are played, while minimizing the worst case is appropriate when a single-game guarantee is required.

Adversarial worst case is easier than Standard worst case. In Evil mode the exact optimum is $D(A) = 4$ guesses. In Standard mode the worst-case optimum is $W(A) = 5$: under optimal play, the hardest fixed Wordle answer still requires 5 guesses. The implication is that the deterministic evil adversary is actually a weaker opponent than the unluckiest possible fixed answer. Evil mode is only harder than Standard mode in expectation (Standard averages 3.421).

Limitations and future work. The main limitation of the benchmark is that I could not implement an exact Standard-mode DP in pure Python within a reasonable time budget. A natural next step would be to vectorize the partition scoring using NumPy, which should give a 5–10× speedup and might make the full DP feasible overnight. A more ambitious next step would be to port the inner loop to a compiled language.

Conclusion

I constructed a benchmark harness for Wordle that evaluates six strategies across three game variants and compares their results to the published optima. The central finding is that simple one-step heuristics perform remarkably well: `expected-entropy` achieves an average of 3.465 guesses in Standard mode, within 0.04 guesses of the optimum of Bertsimas & Paskov (2025), and `posterior-expectimax` attains the optimal Standard-mode worst case of 5 guesses reported by Selby (2022).

A second, perhaps more surprising finding concerns Unknown mode. I expected the mode-aware blend (`posterior-expectimax`) to beat the mode-agnostic `expected-entropy`. It does not: the Bayesian posterior over game type concentrates above 0.99 within two turns, so mode-aware blending only has a brief window in which to matter, and empirically that window is too small to move the average. The best Unknown-mode policy on this corpus is mode-agnostic — a clean negative result for the engineering hypothesis that motivated the blend in the first place.

Evil mode is the one variant where I was able to solve the recurrence exactly in pure Python, because the adversary's deterministic tie-breaking rule collapses the reachable state space to a few thousand nodes. My solver recovers the published bound $D(A) = 4$ with opening guess `RAISE`. The one-guess gap between the DP and every greedy heuristic should not be read as a general statement about the power of dynamic programming; it is a consequence of determinism-induced tractability in this one mode.

Compared to popular Wordle advice, these results suggest that the commonly recommended heuristics (entropy maximization and minimax splitting) are close enough to optimal that the residual gap is not meaningfully exploitable by a human player. The cost of greediness is real but small.

Finally, the benchmark's response to modeling changes is itself informative. The information-theoretic lower bound on expected guesses grows only as $\log_2 N$, so expected depth is relatively insensitive to the size of the answer list. Conversely, if the evil adversary were redefined to prefer the smallest consistent bucket, Evil mode would collapse toward Standard mode and $D(A)$ would drop sharply. Both perturbations are a single configuration change away in the engine, which was one of the original motivations for developing WordleGym as a reusable benchmark rather than a one-off analysis script.

References

Bertsimas, D., & Paskov, A. (2025). An exact solution to Wordle. *Operations Research*, 73(3), 1384–1394. https://doi.org/10.1287/opre.2022.0434. Originally circulated in 2022 as the preprint "An Exact and Interpretable Solution to Wordle" (https://auction-upload-files.s3.amazonaws.com/Wordle_Paper_Final.pdf); accessible summary at https://mitsloan.mit.edu/ideas-made-to-matter/how-algorithm-solves-wordle.

Selby, A. (2022). The best strategies for Wordle. *sonorouschocolate.com*. Available at https://sonorouschocolate.com/notes/index.php?title=The_best_strategies_for_Wordle.

Sanderson, G. (2022). Solving Wordle using information theory. *3Blue1Brown* (YouTube). https://www.youtube.com/watch?v=v68zYyaEmEA. Popularization of the Shannon-entropy heuristic that underlies the `expected-entropy` strategy.

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423.

Wardle, J. (2022). Wordle [game]. *The New York Times*. Canonical answer and allowed-guess lists at https://www.nytimes.com/games/wordle.